Review notes · for you

AI security engineering: unit check

AI security · University · Unit check

Materials
Paper and a pen. No device needed.

Learning goals

- Threat-model an AI application
- The OWASP Top 10 for LLM applications
- Attacking and hardening models
- Red teaming and responsible disclosure

Activity: Design review

1. Teams of three get one made-up AI app
2. Five minutes: its top three risks and their controls
3. Swap: the other team tries to break your plan, on paper
4. Agree the fixes, and the test that proves each one

Everything stays on paper and made-up: no live systems.

Discussion

- Which practice will you use first on your own work?
- Which question surprised you, and why?
- What would you tell a classmate about testing systems without permission?

Check yourself

- My first practice from this unit, and the project I'll use it on

Try it this week
Pick the practice from this unit you'll use first on something you're building (a threat model, an OWASP review, a robustness test or an evaluation harness) and use it this week.

Quiz sheet

AI security engineering: unit check

AI security · University · Unit check

Name _____ Class _____ Date _____

1 **In a threat model of an LLM app, why isn't the model a trust boundary?**

- ◯ It runs on someone else's computer
- ◯ Anything in its context can steer what it says and which tools it calls
- ◯ It is too slow to check its inputs

2 **Match each OWASP risk to an example of it.**

LLM03 Supply chain	• • • •	Its instructions, with a password inside, are read out
LLM05 Improper output handling	• • • •	A look-alike model file with a backdoor
LLM07 System prompt leakage	• • • •	An invented reference ends up in an essay
LLM09 Misinformation	• • • •	A reply pasted into a page runs as script

3 **What does this print?**

```
flows = [("model", "email"),
         ("web", "model")]
reach = {"web"}
for a, b in flows:
    if a in reach:
        reach.add(b)
print("email" in reach)
```

◯ False ◯ True ◯ web

4 **Telling a model "never reveal the password" is enough to keep a password in its system prompt safe.**

◯ True ◯ False

5 **A linear model has weights 3 and -2. What does this print?**

```
w, x, eps = [3, -2], [2, 1], 0.5
s = sum(a * c for a, c in zip(w, x))
drop = eps * sum(abs(a) for a in w)
print(s, s - drop)
```

◯ 4 1.5 ◯ 4 3.5 ◯ 4 6.5

6 **Which changes make it harder to copy a model by querying it? Choose all that apply.**

- [ ] Rate limits for each user
- [ ] Return labels, not exact scores
- [ ] Publishing the model's weights
- [ ] Watching for unusual query patterns

7 **What does a membership inference attack try to learn?**

- ◯ The model's password
- ◯ Whether a person's record was in the training data
- ◯ Which users are online

8 **You found a flaw in someone else's AI system. Put the responsible steps in order.**

- [ ] Check the fix with them
- [ ] Share the lesson, without the exploit
- [ ] Stop, and don't use it again
- [ ] Give them time to fix it
- [ ] Report it privately to the owner

9 **After fixing one finding, you should re-run the whole test bank, not just that case.**

◯ True ◯ False

10 **Complete the two design rules.**

Treat model output as _____ input, and give an agent the _____ tools its job needs.

Word bank: fewest · most · untrusted · trusted

Check your answers

AI security engineering: unit check

AI security · University · Unit check

1 **Anything in its context can steer what it says and which tools it calls**

Instructions hidden in a page, a file or a message all reach the model as text, and it can follow them.

2 **LLM03 Supply chain — A look-alike model file with a backdoor**
LLM05 Improper output handling — A reply pasted into a page runs as script
LLM07 System prompt leakage — Its instructions, with a password inside, are read out
LLM09 Misinformation — An invented reference ends up in an essay

Faked components, unchecked output, secrets in prompts and confident errors are four different risks, each with its own control: verify, escape, keep secrets in code, and check facts.

3 **False**

The loop sees ("model", "email") first, when model isn't reached yet, and only then adds model.

4 **False**

Assume the system prompt will be read. Secrets belong in the app's code or vault, and access decisions in code, never only in instructions to the model.

5 **4 1.5**

The score is 3 × 2 + (-2) × 1 = 4. The worst nudge of size 0.5 lowers it by 0.5 × (3 + 2) = 2.5, to 1.5: still positive, so the answer can't flip at this budget.

6 **Return labels, not exact scores; Rate limits for each user; Watching for unusual query patterns**

Extraction needs many answers with as much detail as possible: labels only, limits and monitoring take that away.

7 **Whether a person's record was in the training data**

Models are often more confident on examples they trained on, so an attacker can guess whether someone's record was used, which alone can reveal something private, such as being a patient at a clinic.

8 **Top to bottom: 4, 5, 1, 3, 2**

Stopping protects real users; the owner hears first and gets time to fix; the lesson is shared only afterwards, and never as a working exploit.

9 **True**

Fixes break other things. Re-running everything, and keeping the fixed case as a regression test, shows the fix worked and nothing else got worse.

10 **Treat model output as untrusted input, and give an agent the fewest tools its job needs.**

Model output can carry anything that was in its context, so escape and check it like any user input.